MACHINE LEARNING WITH SCIKIT-LEARN

Model Evaluation & Cross-Validation

04

Your model says 95%. Is that skill or self-deception? Learn to measure honestly — before you learn more models.

Master's Degree Program · Hands-on with Google Colab

AGENDA

Today's agenda

01 **Why train/test split exists**
the exam analogy, made brutal

02 **Overfitting & underfitting**
watch a tree memorize

03 **Holdout is a lottery**
diabetes R^2 0.36 → 0.59

04 **k-Fold cross-validation**
every point gets a turn

05 **Regression metrics**
MAE · RMSE · R^2

06 **Classification metrics**
confusion · precision · recall · ROC

07 **When accuracy lies**
imbalanced data & fixes

08 **Comparing models properly**
baseline · same folds · mean ± std

BEFORE THE MODELS

First: learn to measure. Then: measure models.

A score you can't trust is worse than no score — it drives every decision you'll make as a data scientist.

Today's models are black boxes

KNN you know (S01).
DecisionTree, LinearRegression, RandomForest arrive in S05–06.
Today they're prediction machines — that's all we need.

What stays constant

No matter the model, the questions are the same:
Did it memorize? Is the number stable?
Good at what, bad at what?

Why before models

From S05 on, you'll meet a new model every week.
Each one hands you a number.
This session teaches you to interrogate it.

RECAP + SHARPEN

Why we split: you can't grade a student on the exam it saw




```
knn1 = KNeighborsClassifier(n_neighbors=1).fit(X_train, y_train)
print(knn1.score(X_train, y_train), knn1.score(X_test, y_test))
```

- **k=1: every point's closest neighbor is itself**
- train accuracy 100% = pure memorization
- test 93.7% = what it really knows
- **the gap between them is the lie**

```
1.0    0.937
```

VOCABULARY

Overfitting & underfitting — the two diseases

Overfitting — memorized the noise

Training score ↑↑, test score ↓

k=1 KNN · a fully-grown tree · too many features

Knows THIS dataset; fails on the next one.
Like a student who drilled last year's exam answers.

Underfitting — missed the pattern

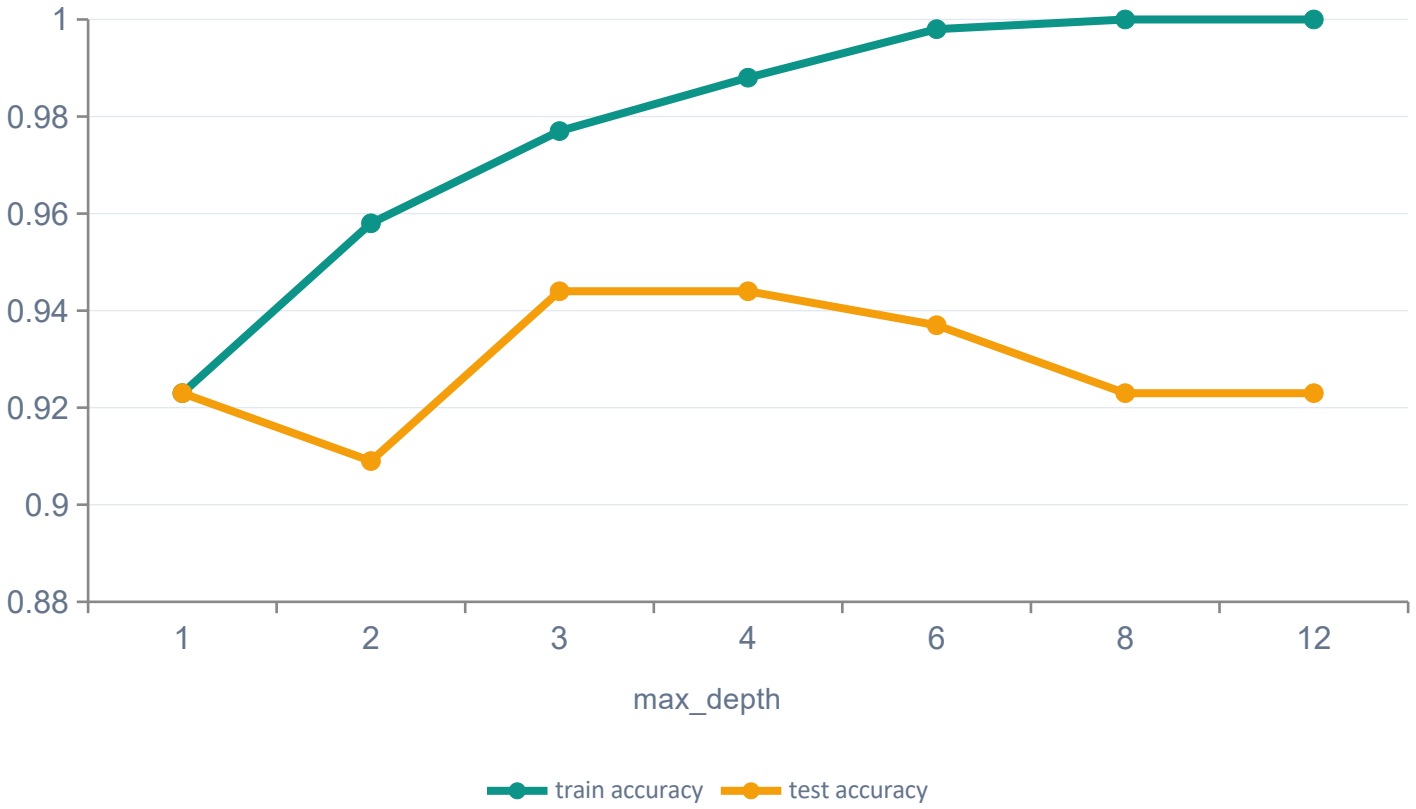
Training score ↓, test score ↓

k=200 KNN · depth-1 tree · too-simple model

Too blunt to capture real structure.
Like a student who only read the chapter titles.

Diagnosis recipe: always print BOTH scores. Gap large → overfit. Both low → underfit. Both high & close → healthy.

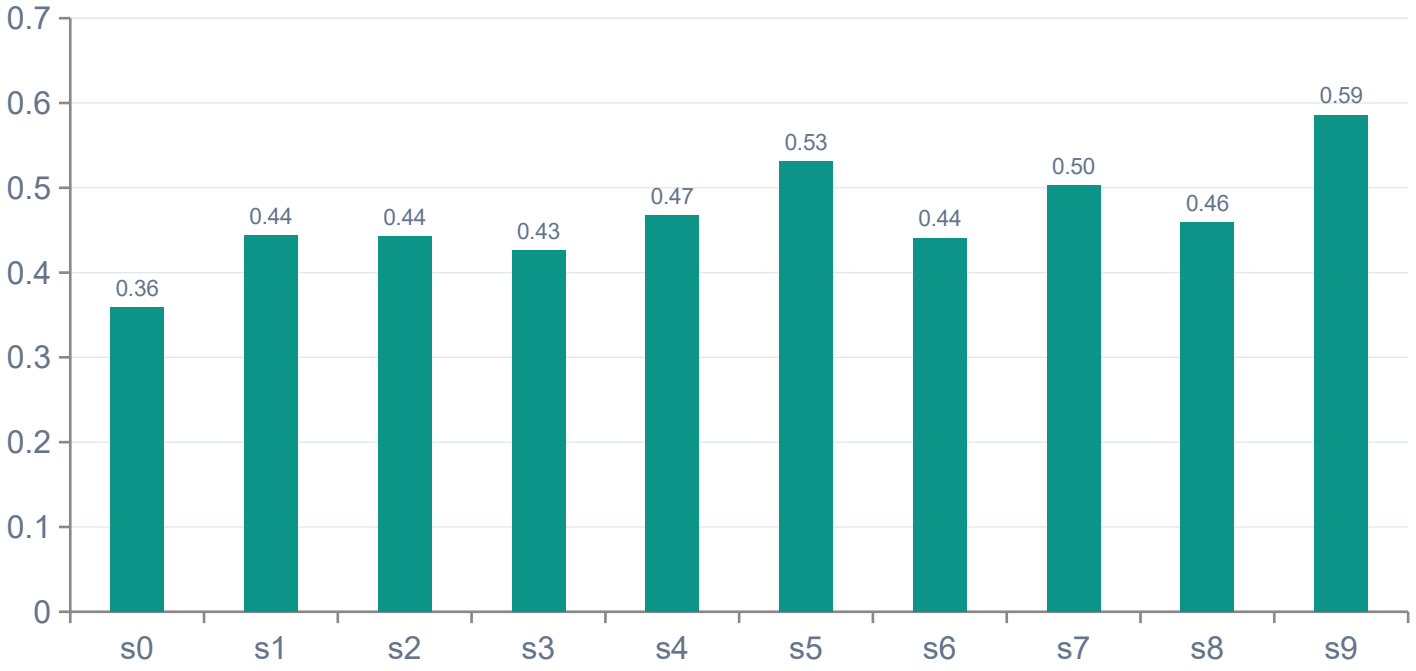
Grow a tree, watch the gap open (breast cancer)



- **Depth 1–3: both rise — learning**
- **Depth 3–4: test peaks (0.944) — sweet spot**
- **Depth 8+: train = 100%, test falls to 0.923**
- the tree starts memorizing individual patients
  - This U-shaped test curve is the bias-variance trade-off you'll tune in S07

Your holdout score is a lottery ticket

Same model, same data, same code — only the random split changes. Ten runs, diabetes dataset:



0.36–0.59

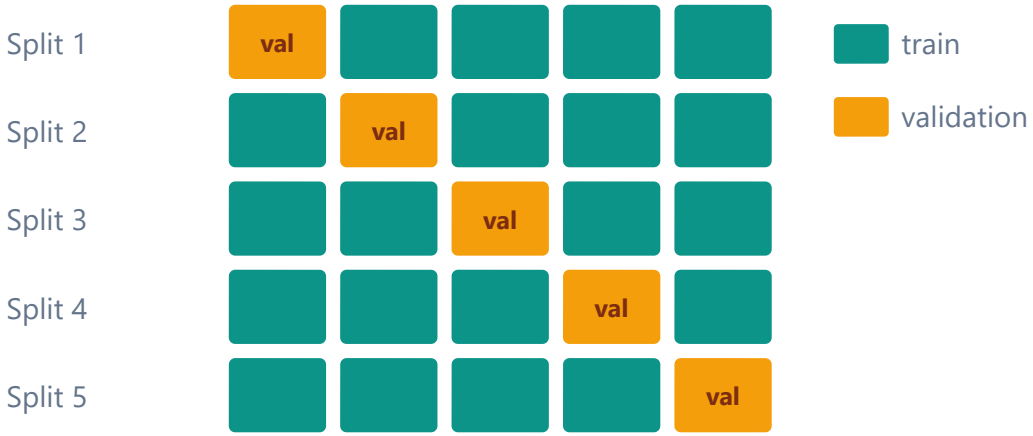
R^2 range across 10 splits
A 44% test-set is not information — it's noise

442 rows is SMALL — and business data often is.
Which number do you report to your boss?

★ THE FIX

k-fold CV: every point takes a turn at being the exam

5-fold cross-validation



- **Split the data into k folds (5 here)**
- train on 4 folds, validate on the 1 left out
- repeat — each fold is the validation once
- **Report mean ± std of the 5 scores**
- mean = honest skill estimate
- std = how much it depends on the split
  - All 442 rows get used for both training AND validation

★ THE FIX

cross_val_score — one line replaces the lottery

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(LinearRegression(), X, y, cv=5)
print(scores, scores.mean(), scores.std())
```

```
[0.430  0.523  0.483  0.426  0.550]
mean 0.482  ±  0.049
```

- **0.482 ± 0.049 — stable & honest**
- compare: holdout swung 0.36 → 0.59
- cross_validate() gives multiple metrics + fit time per fold
  - Cost: trains k times (5× slower) — worth it on small data

Stratified k-fold

For classification: each fold keeps the class ratio of the whole data.
cv=StratifiedKFold(5, shuffle=True, random_state=42)
(Default for classifiers in sklearn — good.)

Where does the test set go?

CV runs on the TRAINING data only. The test set still sleeps untouched until the very end.

How wrong is wrong? MAE · RMSE · R^2

metric	reads as	report when
MAE	average miss, in target units	you must explain to humans
RMSE	average miss, punishing big errors	big misses are extra costly
R^2	fraction of variance explained	comparing across problems

```
LinearRegression on diabetes (holdout):
MAE  41.5   RMSE  53.4   R²  0.485
# target = disease progression, units ≈ points
```

- **"We miss by ±41.5 points on average"**
- MAE is the sentence your stakeholder remembers
- **R^2 = 0.485 → explains ~49% of the variance**
- R^2 of the dummy (predict-the-mean) = 0
- **R^2 can go NEGATIVE — model worse than the mean**

The confusion matrix: four cells of truth

KNN on breast cancer (test set, 143 patients · positive class = malignant)

↑ actually malignant

TRUE POSITIVE 86	**FALSE POSITIVE** 6
FALSE NEGATIVE 4	**TRUE NEGATIVE** 47

← predicted malignant

- **6 false positives**
- healthy patients sent to biopsy — cost: money & fear
- **4 false negatives**
- cancer told "you're fine" — cost: lives
- **Accuracy hides WHICH mistake you make**
- confusion_matrix(y_test, y_pred)

Precision & recall — which error do you fear?

Precision — of my alarms, how many were real?

TP / (TP + FP) = 86/92 = 0.935

High precision = trust the alarm when it rings.
Spam filter: don't bury real mail.

Recall — of the real cases, how many did I catch?

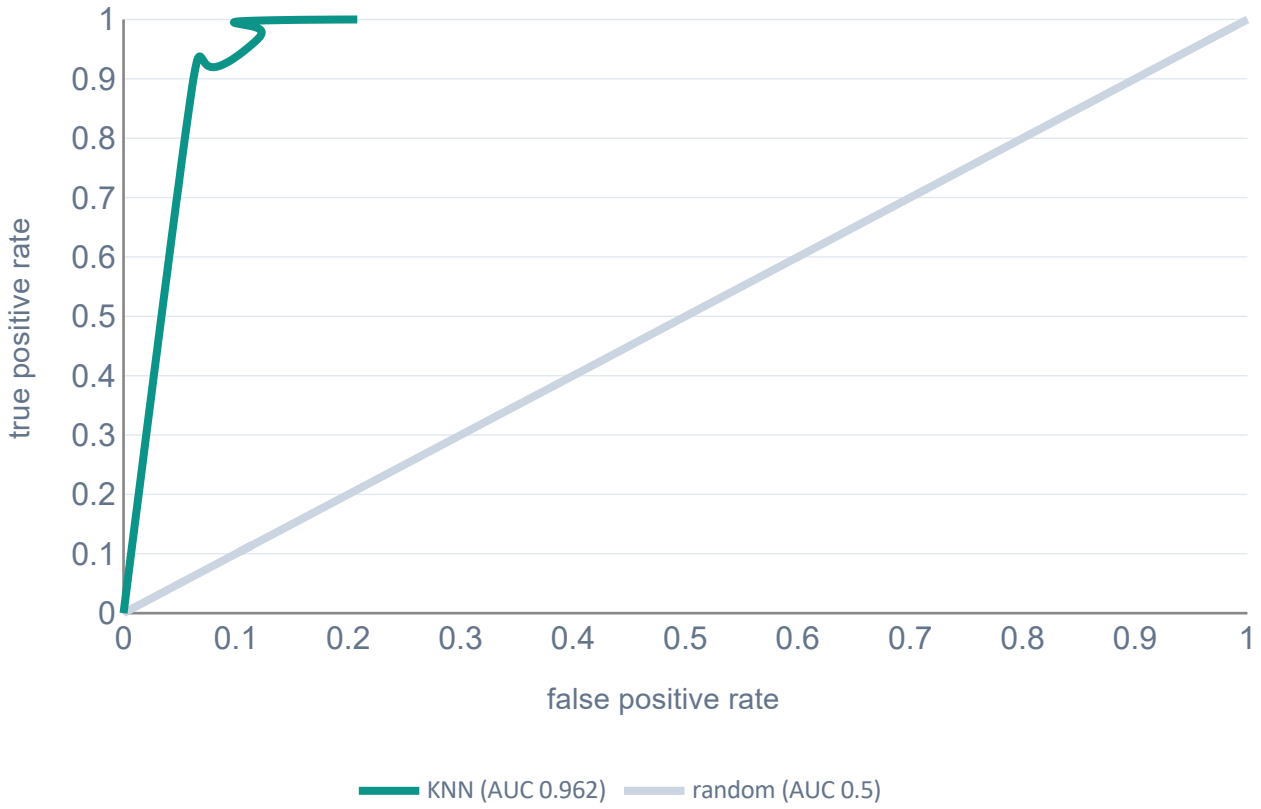
TP / (TP + FN) = 86/90 = 0.956

High recall = few misses.
Cancer screening: catch them all.

business question	protect	metric
Cancer / fraud / safety misses	FN (misses)	RECALL
Spam / arrests / wasted call-centre	FP (false alarms)	PRECISION

F1 = 2·P·R/(P+R) = 0.945 — one number when both matter.

ROC-AUC: judgment across ALL thresholds



- **predict_proba gives scores, not labels**
- threshold 0.5 is ONE operating point on this curve
- move the threshold → trade precision for recall
- **AUC = area under the curve**
- 1.0 = perfect ranking · 0.5 = coin flip
- **AUC is threshold-free — great for comparing models**

THE TRAP

When accuracy lies: imbalanced data

Breast cancer, malignant down-sampled to 19% (like fraud, churn, defects…). Test set is 81% benign.

model	accuracy	recall (malignant)	F1
"nobody is sick" (Dummy most_frequent)	0.810	0.000	0.000
KNN	0.911	0.867	0.788
RandomForest(class_weight="balanced")	0.937	0.867	0.839

- **"81% accurate" while catching ZERO cancers**
- Accuracy answers the wrong question on imbalanced data
- **Fixes: right metric (recall/F1) · class_weight="balanced" · move the threshold**
- **AI assistants default to accuracy — YOU must ask for the right metric**

REFERENCE POINT

Always know the number to beat

model (5-fold CV, breast cancer)	accuracy	ROC-AUC
DummyClassifier (stratified)	0.576	0.536
DummyClassifier (most_frequent)	0.627	0.500
KNN (k=5)	0.935	0.966
RandomForest	0.956	0.989

- **Dummy = the stupidest model**
- guessing at random, or always predicting the majority
- takes 2 lines, answers "is my model even learning?"
- **"0.93 accuracy" alone means nothing — 0.63 is free without any model**
- **Report every score next to its baseline**

The rule

A model is only as good as its margin over a baseline.

Baseline → simple model → tuned model. Each step must EARN its complexity.

(Same philosophy as features in S03: measure, always.)

METHOD

Comparing models the honest way

```
cv = StratifiedKFold(5, shuffle=True, random_state=42)

for name, model in [("dummy", DummyClassifier()),
                    ("knn",  KNeighborsClassifier()),
                    ("rf",   RandomForestClassifier())]:
    acc = cross_val_score(model, X, y, cv=cv)
    auc = cross_val_score(model, X, y, cv=cv,
                          scoring="roc_auc")
    print(name, acc.mean(), auc.mean())
```

- **Three rules baked in:**
- ① same folds for every model (shared cv object) — fair fight
- ② multiple metrics — accuracy AND AUC
- ③ mean ± std over folds — report uncertainty

– Bonus: keep the winning pipeline for S07 tuning

THE PROTOCOL

Where every split lives (the rest of the course)

ALL DATA
split ONCE, first thing

→

TRAIN ~80%
CV lives here: compare & tune

→

TEST ~20%
locked. touched ONCE at the end

CV for decisions

Which model? Which k? Which features? Which threshold?
→ decide on CV scores over the training data.

Test for the final word

After every decision is made: fit on train, score on test ONCE, report it.
Peek twice and it's a training set in disguise.

CHEAT SHEET

Which metric, one slide

situation	primary metric	watch out	in sklearn
Balanced classification	accuracy	—	accuracy_score
Rare positive (fraud, churn, disease)	recall / F1 / PR-AUC	accuracy lies	recall_score, f1_score
Alarms must be trustworthy	precision	recall may sink	precision_score
Ranking / scoring models	ROC-AUC	still check threshold	roc_auc_score
Regression, explainable	MAE	outliers unpunished	mean_absolute_error
Regression, big misses costly	RMSE	sensitive to outliers	mean_squared_error

classification_report(y_test, y_pred) prints precision · recall · F1 per class in one call.

LAB 04 · DO IT IN GOOGLE COLAB

วัดให้จริง — **Overfit, lottery** และ **metric** ที่ถูกต้อง

YOUR MISSION (โจทย์ภาษาไทยใน Colab)

- breast cancer: เทรน KNN k=1 — เทียบ train vs test accuracy และอธิบายช่องว่าง
- วาดกราฟ train/test accuracy ของ DecisionTree ที่ max_depth 1,2,3,4,6,8,12 — หา sweet spot
- diabetes: รัน holdout 10 random_state ต่างกัน — ช่วง R^2 กว้างแค่ไหน? แล้วแก้ด้วย cross_val_score (report mean±std)
- เทียบ Dummy vs KNN vs RandomForest ด้วย StratifiedKFold เดียวกัน — accuracy + ROC-AUC
- อ่าน confusion matrix + precision/recall/F1/AUC ของ KNN แล้วตอบ: ถ้าเป็นหมอ ควรแลก precision กับ recall ยังไง

Learning goal

Reproduce every experiment of today on real data: overfit curves, holdout instability, cross-validation, and the full metric toolbox.

Vibe coding corner

ลองถาม AI: “โจทย์ของฉันคือตรวจมะเร็งจากข้อมูลที่ positive แค่ 19% — ควรใช้ metric อะไร และเพราะอะไร accuracy ถึงไม่ใช่คำตอบ” แล้วเทียบคำตอบกับ cheat sheet หน้าสุดท้าย

KEY TAKEAWAYS

1. Print train AND test scores: a big gap means memorization (KNN k=1: 1.00 train / 0.94 test).
2. A single holdout score is noise — diabetes R^2 swung 0.36→0.59 across splits.
3. k-fold CV turns noise into mean ± std; use StratifiedKFold for classification.
4. Read the confusion matrix: precision guards false alarms, recall guards misses; imbalanced data makes accuracy a lie.
5. Always compare against a Dummy baseline, on the same folds, with the metric that matches the business question.

NEXT

Session 05 — Regression Models — opening the first black boxes